ASCOM ALPACA on the ZWO Seestar S50: A Technical Reference

March 2026

Abstract

The ZWO Seestar S50 is a compact, fully automated smart telescope that originally exposed a proprietary JSON-RPC 2.0 control interface over TCP. Beginning with firmware version 3.0, ZWO introduced cryptographic authentication on this interface, effectively locking out all third-party control software. Simultaneously, ZWO added a native ASCOM ALPACA HTTP REST server on TCP port 32323, providing a standards-based alternative for external motor control. This document catalogues the findings of systematic protocol analysis and empirical testing of both interfaces on firmware 3.1.1, and characterizes the capabilities and limitations of the ALPACA implementation on the Seestar S50.

1. Background: The JSON-RPC Interface (Port 4700)

1.1 Protocol Overview

The Seestar S50 exposes a JSON-RPC 2.0 interface over a persistent TCP connection on port 4700. Messages are newline-delimited (`\r\n`), with one complete JSON object per line. The interface is bidirectional: the client sends commands and receives both solicited responses (keyed by message ID) and unsolicited push events.

A dedicated reader thread is required on the client side. The scope pushes events asynchronously — `PiStatus` (temperature, battery, charger state), `ScopeTrack` (motion state), `FocuserMove`, `RecordingStart`/`RecordingStop`, viewing mode transitions, and `Client` (master control status). Without a reader thread continuously draining the socket, responses are lost, push events queue silently, and all synchronous commands eventually timeout.

1.2 Connection Handshake

The connection sequence is non-trivial and must occur in the correct order:

1. **UDP broadcast** — A `scan_iscope` JSON-RPC message is broadcast to `255.255.255.255:4720` before the TCP connection. This satisfies the scope's "guest mode" handshake; without it, the scope treats the TCP client as a ghost and silently ignores commands.

2. **TCP connect** — Standard socket connection to port 4700.
3. **Initialization sequence** — A series of fire-and-forget commands:
   - `set_user_location` (latitude, longitude, `force=true`) — syncs observer coordinates.
   - `pi_set_time` (year, month, day, hour, minute, second, `time_zone="UTC"`) — syncs the real-time clock.
   - `pi_is_verified` — session verification.
   - `set_setting {"master_cli": true}` — claims master control.
   - `set_setting {"cli_name": "<identifier>"}` — client identification.
4. **Heartbeat** — A keep-alive `pi_is_verified` message must be sent every ~3 seconds to prevent the scope from dropping the connection.

1.3 Master Control

The Seestar firmware supports only one master client at a time. Without master status, the scope accepts motor commands syntactically but silently discards them — no error is returned. The scope announces master status via a push `Client` event containing `{"is_master": true|false}`. If the Seestar iPhone app is open (even in background), it typically holds master, and the third-party client must reclaim it by re-sending `set_setting {"master_cli": true}`.

1.4 The `verify` Parameter

Firmware versions introduced a `verify` parameter with version-dependent injection rules:

Firmware	Parameter format	Rule
<2706, dict params	Dict	Inject `"verify": true` into the params dict
>=2706, dict params	Dict	Omit `verify` entirely (error code 109 if present)
Any, list params	List	Append the string `"verify"` to the list
Any, no params	—	Add top-level `"verify": true`

This is a firmware handshake mechanism, not cryptographic authentication in the traditional sense.

1.5 Command Taxonomy

The JSON-RPC interface exposes commands across several domains:

Viewing modes (fire-and-forget; confirmed by push events): - `iscope_start_view` — Enter solar (`mode: "sun"`), lunar (`mode: "moon"`), scenery (`mode: "scenery"`), or star (`mode: "star"` with `target_ra_dec`) mode. - `iscope_stop_view` — Exit current viewing mode.

Recording (fire-and-forget; confirmed by `RecordingStart`/`RecordingStop` events): - `start_record_avi {"raw": false}` — Begin MP4 recording. - `stop_record_avi` — End recording.

Motor control (synchronous or fire-and-forget): - `scope_speed_move` — Nudge at a given speed (0–8000 scale), angle (with +180° firmware offset), and duration. - `scope_open` — Open (unfold) the telescope arm. - `scope_park` — Park (fold) the arm.

Camera and optics (synchronous): - `start_auto_focuse` — Autofocus (note: typo in firmware command name). - `move_focuser` — Manual focus step. - `set_control_value ["gain", N]` — Set sensor gain. - `set_setting {"exp_ms": ...}` — Set exposure time. - `set_setting {"stack_lenhance": bool}` — Light pollution filter toggle. - `pi_output_set2 {"heater": ...}` — Dew heater control.

Telemetry queries (synchronous, require reader thread): - `scope_get_horiz_coord` — Alt/Az readout. - `scope_get_equ_coord` — RA/Dec readout. - `get_view_state` — Current viewing mode. - `get_device_state` — Device state summary.

1.6 Firmware 3.0+ Lockout

Testing on firmware 3.1.1 (Seestar App v3.0+) revealed the following behavioral changes:

Observation	Evidence
TCP port 4700 still accepts connections	Socket stays `ESTABLISHED`
Scope sends `PiStatus` push events (temperature, battery)	Received by reader thread
All query/response commands timeout (5–8 s)	Tested: `scope_get_horiz_coord`, `get_view_state`, `scope_get_equ_coord`, `get_device_state`, `pi_is_verified`, `get_setting`
Scope sends zero responses to any command	Raw socket reader receives nothing back
`scope_speed_move` produces no physical movement	Tested at speed=4000, dur=10 s

Observation	Evidence
UDP port 4720 accepts broadcast but sends no reply	No connection refused; no response
`iscope_start_view mode=sun` appears functional	Scope tracks sun on startup; unclear if command or default behavior

The root cause, confirmed by the maintainer of the seestar_alp project (GitHub issue #697), is that ZWO intentionally added cryptographic authentication to the port 4700 interface. Third-party applications can connect and receive temperature telemetry, but all motor commands and query responses are silently suppressed.

What still works on port 4700 (firmware 3.0+): - TCP connection and push event reception (`PiStatus`, `ScopeTrack`, `FocuserMove`, etc.) - Viewing mode commands (`iscope_start_view`, `iscope_stop_view`) — fire-and-forget, confirmed via push events - Recording commands (`start_record_avi`, `stop_record_avi`) — fire-and-forget, confirmed via events - Camera settings (gain, exposure, focus, dew heater) — fire-and-forget

What no longer works on port 4700: - All query/response commands (position readout, state queries) - Motor movement (`scope_speed_move`, `scope_open`, `scope_park`) - Any command requiring a synchronous response

2. The ALPACA Interface (Port 32323)

2.1 Protocol Overview

ASCOM ALPACA is a platform-independent REST/JSON standard for astronomy device control, maintained by the ASCOM Initiative. The Seestar S50 firmware 3.0+ ships a native ALPACA server that exposes the ASCOM `ITelescope` interface over HTTP on TCP port 32323.

The protocol is stateless HTTP: properties are read via `GET`, commands are executed via `PUT` with `application/x-www-form-urlencoded` bodies. Every request includes `ClientID` and `ClientTransactionID` parameters; every response includes these plus a `ServerTransactionID`, an `ErrorNumber` (0 = success), and an `ErrorMessage`.

No authentication or encryption is used. The server binds to all interfaces.

2.2 Discovery

ALPACA defines a UDP discovery protocol. A client broadcasts the ASCII string `alpacadiscovery1` to `255.255.255.255:32227`. ALPACA-capable devices reply with a JSON payload containing at minimum the `AlpacaPort` field:

```
→ UDP broadcast to 255.255.255.255:32227: "alpacadiscovery1"
← Reply from 192.168.x.x:  {"AlpacaPort": 32323}
```

This is distinct from the proprietary ALP discovery on port 4720.

2.3 Management Endpoints

Three management endpoints exist outside the device API namespace:

Endpoint	Returns
`GET /management/v1/description`	Server name, manufacturer, version
`GET /management/v1/configureddevices`	Array of device type/number/name/uniqueid entries
`GET /management/apiversions`	Supported API version integers

On the Seestar, `/management/v1/configureddevices` returns a single telescope device at index 0.

2.4 API Structure

All telescope endpoints live under:

```
http://{host}:32323/api/v1/telescope/0/{endpoint}
```

GET requests pass parameters as query strings:

```
GET /api/v1/telescope/0/canmoveaxis?Axis=0&ClientID=1&ClientTransactionID=5
```

PUT requests pass parameters as form-encoded bodies:

```
PUT /api/v1/telescope/0/moveaxis
Content-Type: application/x-www-form-urlencoded

Axis=0&Rate=1.0&ClientID=1&ClientTransactionID=6
```

Response format (all endpoints):

```
{
  "Value": true,
  "ErrorNumber": 0,
  "ErrorMessage": "",
  "ClientTransactionID": 5,
  "ServerTransactionID": 42
}
```

2.5 Capability Profile

The Seestar S50 reports the following capabilities via ALPACA property reads:

Capability	Value	Implication
`canslew`	`true`	Synchronous RA/Dec GoTo supported
`canslewasync`	`true`	Asynchronous RA/Dec GoTo supported
`canslewaltaz`	**`false`**	No native Alt/Az GoTo — clients must convert to RA/Dec
`canslewaltazasync`	**`false`**	Same limitation
`canmoveaxis`	`true`	Rate-based axis nudge supported (both axes)
`canpark`	`true`	Park/unpark (arm fold/unfold) supported
`canpulseguide`	**`false`**	No autoguider pulse guide
`cansettracking`	`true`	Sidereal tracking toggle supported
`interfaceversion`	`3`	ASCOM ALPACA interface version 3

The `canslewaltaz=false` limitation is significant. To slew to a known altitude and azimuth, the client must perform the spherical trigonometric conversion to equatorial coordinates using the observer's latitude, longitude, and the current Greenwich Apparent Sidereal Time.

2.6 Confirmed Working Operations

Tested on firmware 3.1.1 with the scope in station mode (connected to home WiFi):

Position readout: - `GET rightascension` — RA in hours [0, 24) - `GET declination` — Dec in degrees [-90, +90] - `GET altitude` — Alt in degrees - `GET azimuth` — Az in degrees [0, 360) - `GET siderealtime` — Local Sidereal Time in hours - `GET utcdate` — Scope clock as ISO 8601 string

Motor control: - `PUT moveaxis` with `Axis=0, Rate=1.0` — Primary axis (RA/Az) nudge at 1 deg/s. Rate=0 stops. Both axes (0 and 1) confirmed independently. Motor continues at constant rate until a Rate=0 stop command is issued. - `PUT slewtocoordinatesasync` with `RightAscension=<hours>, Declination=<degrees>` — Asynchronous GoTo. Returns

immediately; poll `GET slewing` to detect completion. Confirmed: scope physically moves to target coordinates. - `PUT abortslew` — Halts any in-progress slew.

Tracking: - `PUT tracking` with `Tracking=true|false` — Enables or disables sidereal tracking. Confirmed working. - `GET tracking` — Returns current tracking state.

Park/unpark: - `PUT park` — Parks (folds arm). Confirmed. - `PUT unpark` — Unparks (opens arm). Confirmed. - `GET atpark` — Returns boolean park state.

Connection: - `PUT connected` with `Connected=true` — Opens ALPACA session. No master control negotiation required (unlike JSON-RPC). - `PUT connected` with `Connected=false` — Closes session.

2.7 Operations Not Available via ALPACA

The Seestar's ALPACA server exposes only the standard ASCOM `ITelescope` interface. The following Seestar-specific functions have no ALPACA equivalent:

Function	Notes
Solar/lunar/scenery viewing modes	Proprietary; use JSON-RPC `iscope_start_view`
Video recording (AVI/MP4)	No ASCOM `ICamera` device exposed
Photo capture	No camera device
Sensor gain and exposure control	No camera device
Autofocus	No ASCOM `IFocuser` device exposed
Manual focus stepping	No focuser device
Dew heater control	No ASCOM `ISwitch` device exposed
Light pollution filter toggle	Proprietary setting
RTSP live preview stream	Proprietary; accessible at `rtsp://{host}:4554/stream`
File management (album listing, download)	Proprietary HTTP endpoints on scope
Device telemetry (CPU temp, battery, charger)	Proprietary push events on port 4700

The ASCOM standard defines an `ITelescopeV4.Action()` method for device-specific extensions, but testing has not confirmed whether ZWO exposes any proprietary actions through this endpoint.

2.8 Alt/Az to RA/Dec Conversion

Since `canslewaltaz` is false, clients must convert altitude/azimuth to right ascension/declination before issuing a GoTo. The conversion uses standard spherical trigonometry:

```
sin(Dec) = sin(Alt) * sin(Lat) + cos(Alt) * cos(Lat) * cos(Az)
Dec = arcsin(sin(Dec))

cos(HA) = (sin(Alt) - sin(Lat) * sin(Dec)) / (cos(Lat) * cos(Dec))
HA = arccos(cos(HA))
if sin(Az) > 0:  HA = 2π - HA

LST = GAST + Longitude/15
RA = LST - HA * 12/π   (mod 24)
```

Where `Lat` is the observer's latitude, `GAST` is Greenwich Apparent Sidereal Time, `HA` is the hour angle, and `LST` is the local sidereal time. All intermediate calculations are in radians; RA is output in hours, Dec in degrees.

3. Hybrid Architecture

Given the complementary capabilities of the two interfaces, a practical control architecture for the Seestar S50 on firmware 3.0+ uses both simultaneously:

Domain	Interface	Port	Protocol
Motor control (GoTo, nudge, stop)	ALPACA	32323	HTTP REST
Tracking enable/disable	ALPACA	32323	HTTP REST
Park / unpark	ALPACA	32323	HTTP REST
Position readout (RA/Dec/Alt/Az)	ALPACA	32323	HTTP REST
Viewing modes (solar, lunar, scenery)	JSON-RPC	4700	TCP
Recording start/stop	JSON-RPC	4700	TCP
Camera settings (gain, exposure, focus)	JSON-RPC	4700	TCP
Device telemetry (temp, battery)	JSON-RPC	4700	TCP push events
RTSP live preview	Proprietary	4554	RTSP/TCP
Discovery	Both	4720 (ALP) / 32227 (ALPACA)	UDP broadcast

The JSON-RPC connection must be maintained for its push event stream (the only source of recording state, viewing mode changes, and device telemetry) and for issuing viewing mode and recording commands. The ALPACA connection handles all motor control — the only domain where JSON-RPC is now locked out.

4. Operational Requirements and Constraints

4.1 Network Mode

ALPACA only functions when the Seestar is in **station mode** — connected to a local WiFi network. In AP mode (the scope's own hotspot), port 32323 is not exposed. Station mode is configured through the Seestar iPhone/Android app.

4.2 Arm State

The telescope arm must be opened (unparked) before motor commands will execute. This can be done via the `unpark` ALPACA command or through the Seestar app. Attempting `moveaxis` or `slewtocoordinatesasync` while parked returns an ALPACA error.

4.3 No Authentication

Neither interface employs network-level authentication. The ALPACA server binds to all interfaces on port 32323 without TLS or access control. Any device on the same network can issue commands. Deployments should restrict network access accordingly.

4.4 Polling vs Push

ALPACA is strictly request/response — there is no push event mechanism. Clients must poll for state changes (position, slewing status, tracking state). A 2–3 second polling interval is practical. JSON-RPC, by contrast, pushes events asynchronously and requires a persistent reader thread.

5. Summary of Protocol Differences

Aspect	JSON-RPC (Port 4700)	ALPACA (Port 32323)
Transport	Persistent TCP socket	Stateless HTTP
Encoding	JSON-RPC 2.0, newline-delimited	REST/JSON, form-encoded PUT bodies
Discovery	UDP broadcast, port 4720	UDP broadcast, port 32227
Authentication	`verify` param (handshake, not crypto)	None
Session	Master control claim required	`connected=true` sufficient

Aspect	JSON-RPC (Port 4700)	ALPACA (Port 32323)
managemen t		
Heartbeat	Required (3 s `pi_is_verified`)	Not required
Push events	Yes (PiStatus, ScopeTrack, etc.)	No — polling only
Motor control (fw 3.0+)	Silently dropped	Fully functional
Viewing modes	Functional	Not exposed
Recording	Functional	Not exposed
Camera/foc user/heater	Functional	Not exposed
Position readout (fw 3.0+)	Timeout (queries blocked)	Fully functional

References

1. ASCOM ALPACA API specification — https://ascom-standards.org/api/
2. seestar_alp — Open-source Seestar controller: https://github.com/smart-underworld/seestar_alp
    - Key source: `device/seestar_device.py` — `guest_mode_init()`, `transform_message_for_verify()`, `is_client_master()`, `move_scope()`
    - Issue #697: Confirmation of firmware 3.0+ lockout
3. ZWO Seestar S50 product page — https://www.zwoastro.com/seestar/